

1. Ce reprezinta sistemele de baze de cunostinte?

O bază de cunoștințe poate fi privită ca o colecție de entități având o structură sintactică în concordanță cu o anumită metodă de reprezentare a cunoștințelor. Astfel, prin efectuarea unui calcul numit **raționament** sau **inferență**, se pot deduce noi informații cu privire la obiectele sau conceptele reprezentate în bază. Cunoștințele dintr-un sistem inteligent descriu, de obicei, un anumit domeniu, care devine astfel **universul problemei**, și formează conținutul bazei de cunoștințe.

Tipuri de cunostinte:

1. Cunoștințe relaționale simple
2. Cunoștințe care permit moștenirea
3. Cunoștințe inferențiale
4. Cunoștințe procedurale

2. Concepte de inferenta.

Inferența este procesul prin care se obțin **informații noi** pe baza informațiilor deja existente într-o **bază de cunoștințe**, folosind **reguli logice**.

O bază de cunoștințe este alcătuită din:

- **Fapte** – informații cunoscute ca adevărate
- **Reguli** – afirmații de tip *dacă-atunci*

Inferența

Inferența aplică regulile asupra faptelor pentru a obține **concluzii noi**.

Exemplu:

- Fapt: Ion este student
- Regulă: Dacă cineva este student, atunci are carnet de student
- Concluzie: Ion are carnet de student

Tipuri de inferență

- **Inferență deductivă** – de la general la particular (concluzii sigure)
- **Inferență inductivă** – de la particular la general (concluzii probabile)
- **Inferență abductivă** – determină cea mai probabilă explicație

Rolul inferenței

Inferența permite:

- obținerea de cunoștințe noi,
- luarea deciziilor,
- rezolvarea problemelor,
- funcționarea sistemelor expert.

3. Rationamentul FORWARD si BACKWARD

Raționament Forward (Forward Chaining)

Raționamentul forward este o metodă de inferență **condusă de date**, care pornește de la **faptele cunoscute** și aplică reguli pentru a obține **noi concluzii**, până când se ajunge la un scop sau nu mai pot fi generate fapte noi.

Caracteristici

- pornește de la **date inițiale**;
- aplică reguli de tip **IF–THEN**;
- generează **toate concluziile posibile**;
- este potrivit când avem **multe date** și nu un scop clar.

Exemplu

Reguli:

- $IF A \wedge B \rightarrow C$
- $IF C \rightarrow D$

Fapte inițiale:

- A, B

Inferență:

- $A \wedge B \rightarrow C$
- $C \rightarrow D$

Concluzie: **D**

Raționament Backward (Backward Chaining)

Raționamentul backward este o metodă de inferență **condusă de scop**, care pornește de la **o concluzie dorită** și caută faptele și regulile care pot susține acea concluzie.

Caracteristici

- pornește de la **scop**;

- caută reguli care duc la scop;
- verifică dacă **premisele sunt adevărate**;
- este eficient când avem **puține scopuri bine definite**.

Exemplu

Scop:

- D

Reguli:

- IF $C \rightarrow D$
- IF $A \wedge B \rightarrow C$

Căutare:

- pentru D trebuie C
- pentru C trebuie A și B
- A și B sunt adevărate

Scopul **D** este demonstrat.

4. Frame-uri

Prin ce se caracterizeaza un obiect?

Un **obiect (cadru)** se caracterizează prin:

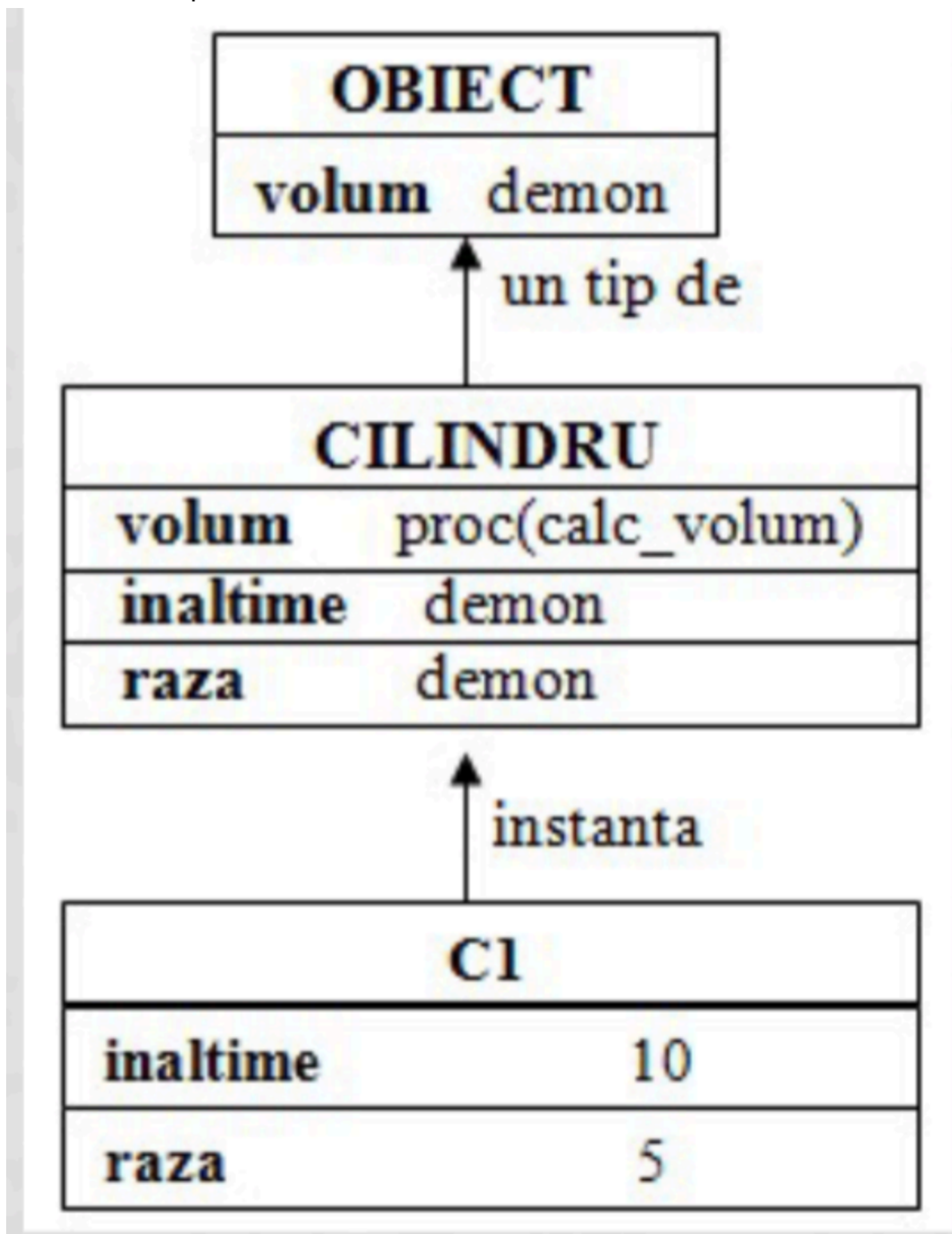
- faptul că este o **structură de date** pentru reprezentarea cunoștințelor;
- conține **slot-uri (attribute)**;
- fiecare slot are **valori asociate**;
- valorile pot fi:
 - valori simple,
 - liste de valori,
 - **proceduri** care calculează valori pe baza altor informații;
- asupra slot-urilor se pot defini **constrângeri**;
- obiectele pot reprezenta atât entități concrete, cât și concepte abstracte (clase, evenimente, situații).

Demon

Demon este:

- un **tip special de valoare pentru sloturi**;
- indică faptul că obiectul/cadrul este **prea generic** pentru a avea o valoare concretă pentru acel atribut;

- semnifică faptul că **valoarea atributului va fi specificată la descendenți**;
- dacă un slot are valoarea *demon*, atunci **instanțele sau subclasele trebuie să furnizeze valori directe** pentru acel atribut.



Unknown

Unknown apare ca rezultat al funcției de răspuns și semnifică:

- faptul că **atributul cerut nu este definit** în baza de cunoștințe pentru cadrul respectiv;
- perechea (*cadru, atribut*) **nu aparține** mulțimilor pentru care există valori directe sau valori calculate;
- nu există informație disponibilă nici direct, nici prin moștenire.

Undefined

Undefined apare atunci când:

- valoarea unui atribut ar trebui **calculată printr-o procedură**;
- **nu pot fi determinate valorile atributelor de intrare** necesare procedurii;
- calculul nu poate fi realizat, deci rezultatul este **nedefinit**.

Moștenire

Moștenirea este:

- **principiul fundamental de inferență** în sistemele de reprezentare cu cadre;
- bazată pe ideea că **obiectele similare au trăsături comune**;
- realizată prin organizarea obiectelor în **clase și ierarhii**;
- permite ca un cadru să **moștenească proprietăți** de la cadrele mai generale (superclase);
- căutarea unei valori se face trecând la **cel mai apropiat cadru mai general** conform relațiilor *este-un / un_fel_de*;
- poate fi:
 - **moștenire unică** (un singur părinte),
 - **moștenire multiplă** (mai mulți părinți, posibil ambiguități).

5. Sistemele Lindenmayer

Deterministe

Într-un sistem Lindenmayer determinist (DOL), „determinist” înseamnă că fiecărui simbol îi corespunde exact o singură regulă de rescriere.

Cel mai simplu L-Sistem:

determinist și context-free.

Exemplu:

axiomă: b

reguli de rescriere:

$a \rightarrow ab$,

$b \rightarrow a$



Exemplu de derivări într-un sistem
DOL

Desen turtle(o rotatie)

```
import turtle

t = turtle.Turtle()

for i in range(3):
    t.forward(120)
    t.left(120)    # rotație la stânga

turtle.done()
```

6. Reguli

```
class Mama {
public:
    string mama;
    string copil;
};

class Frate {
public:
    string frate1;
    string frate2;
};

class Sot {
public:
    string sot1;
    string sot2;
};

class Copil {
public:
    string copil;
    string parinte;
};

const int MAX_FAPTE = 100;

class Bunic {
public:
    string bunic;
    string nepot;
```

```

};

class Ver {
public:
    string ver1;
    string ver2;
};

Mama listaMame[MAX_FAPTE];
    Sot listaSoti[MAX_FAPTE];
    Frate listaFratii[MAX_FAPTE];
    Copil listaCopii[MAX_FAPTE];
    Bunic listaBunici[MAX_FAPTE];
    Ver listaVeri[MAX_FAPTE];

    int nMame = 0;
    int nSoti = 0;
    int nFratii = 0;
    int nCopii = 0;
    int nBunici = 0;
    int nVeri = 0;

// Fapte introduse manual: cine e mama cui si cine cu cine e sot/sotie.
listaMame[nMame++] = {"Roberta", "Costin"};
listaMame[nMame++] = {"Roberta", "Ana"};
listaMame[nMame++] = {"Cosmina", "Andrei"};
listaMame[nMame++] = {"Cosmina", "Cristina"};

listaSoti[nSoti++] = {"Roberta", "Gheorghe"};
listaSoti[nSoti++] = {"Cosmina", "Catalin"};

listaMame[nMame++] = {"Maria", "Roberta"};
listaSoti[nSoti++] = {"Maria", "Petru"};
listaMame[nMame++] = {"Lidia", "Cosmina"};
listaSoti[nSoti++] = {"Lidia", "Sorin"};

listaMame[nMame++] = {"Maria", "Sorina"};
listaSoti[nSoti++] = {"Sorina", "Mihai"};
listaMame[nMame++] = {"Sorina", "Paul"};
listaMame[nMame++] = {"Sorina", "Daria"};

listaMame[nMame++] = {"Elena", "Mara"};
listaMame[nMame++] = {"Elena", "Vlad"};
listaSoti[nSoti++] = {"Elena", "Dan"};

```

```
listaMame[nMame++] = {"Nora", "Elena"};
listaSoti[nSoti++] = {"Nora", "Danut"};

listaMame[nMame++] = {"Irina", "Bogdan"};
listaSoti[nSoti++] = {"Irina", "Tudor"};

listaMame[nMame++] = {"Olga", "Irina"};
listaSoti[nSoti++] = {"Olga", "Mircea"};
```